DOCUMENTAÇÃO TÉCNICA

# CRUD administrativo em Java com Spring MVC JPA JDBC e criptografia

Projeto Mottainai

| | |
|---|---|
| **Responsável** | João Victor Rodrigues |
| **Disciplina** | Programação Orientada a Objetos (POO) |
| **Equipe / colaboradores** | João Victor Rodrigues, Pedro Henrique Ângelo e Miguel Quadros Araújo |
| **Versão** | 1.1 \| 29/10/2026 |

## Finalidade

Este documento apresenta a documentação final de Programação Orientada a Objetos do Projeto Mottainai. Descreve o CRUD administrativo em Java, sua arquitetura em camadas, a integração com PostgreSQL e JSP, as regras de segurança e os registros de testes.

# 1. Contexto e objetivo

O CRUD administrativo é o sistema de cadastro responsável por cadastrar, consultar, atualizar e excluir os dados administrativos do Mottainai, uma solução de gestão preditiva de estoque para o varejo. A aplicação organiza essas operações em classes Java com responsabilidades separadas e apresenta os dados por uma interface web.

Spring MVC recebe as requisições HTTP e retorna páginas JSP ou respostas REST. Os Services aplicam as regras de negócio; as interfaces do pacote dao estendem JpaRepository para acessar PostgreSQL. O login administrativo também possui uma consulta JDBC específica com JdbcTemplate.

## Responsabilidade individual

João Victor Rodrigues é responsável por implementar as classes do sistema, desenvolver Model, DTO, Service e Controller, validar as entradas, tratar exceções e documentar os testes e as evidências de entrega.

Pedro Henrique Ângelo é responsável pela integração com PostgreSQL via JDBC e pelas estruturas de acesso a dados em Java puro. Miguel Quadros Araújo é responsável pela revisão dos códigos e pela integração das requisições HTTP e das páginas JSP com o backend Spring MVC.

## Decisão técnica

A implementação utiliza Java 21, Spring Boot 4.0.0 e Gradle, conforme a configuração de build.gradle. A arquitetura reúne Spring MVC para requisições e páginas, JPA para persistência e JDBC para a consulta das credenciais administrativas.

## Relação com as demandas do projeto

| Disciplina | Exigência | Como esta entrega contribui |
|---|---|---|
| POO | CRUD Java de pelo menos três tabelas e integração web/banco. | Oito recursos administrativos, organizados em camadas. |
| LPR | Lógica, validação e tratamento de erros. | Métodos Java verificam entradas e regras do cadastro. |
| Banco de Dados 1 | Modelo relacional e integração SQL. | Mapeamentos JPA das tabelas PostgreSQL. |
| Sistemas Operacionais | Configuração e variáveis de ambiente. | Banco e chave AES configurados pelo ambiente. |
| MPTI e entrega geral | Planejamento e evidências. | Relação entre classes, testes e comprovações. |

# 2. Especificações da disciplina

| Tecnologia / ferramenta | Uso no projeto |
|---|---|
| Java 21 | Classes, interfaces, métodos, coleções e exceções. |
| Spring Boot 4.0.0 / Gradle | Inicialização, dependências, build e testes. |
| Spring MVC / JSP / JSTL | Rotas REST, formulários e páginas dinâmicas. |
| JPA / Hibernate / PostgreSQL | Mapeamento e persistência por JpaRepository. |
| JDBC / JdbcTemplate | Consulta parametrizada das credenciais. |
| Bean Validation / Lombok | Validação de DTOs e métodos de acesso. |
| AES 256 GCM / BCrypt | Cifra reversível dos dados e hash de senhas. |
| JUnit / Mockito / MockMvc | Testes de regras, contratos HTTP e sessão. |

# 3. Parte técnica

O fluxo das telas é JSP → ViewController → Service → DAO → PostgreSQL. As rotas REST utilizam Controller → Service → DAO e retornam DTOs. A persistência principal é JPA, embora as interfaces tenham o sufixo Dao.

| Componente / artefato | Responsabilidade | Evidência no código |
|---|---|---|
| model | Representar entidades e vínculos. | Company, Store e Employee. |
| dto | Separar entrada e saída da entidade. | RequestDTO e ResponseDTO. |
| controller | Receber HTTP e retornar REST ou página. | CompanyController e ViewController. |
| service | Aplicar regras e preparar DTOs. | CompanyService e EmployeeService. |
| dao | Consultar e persistir por JPA. | CompanyDao extends JpaRepository. |
| security | Cifrar dados, gerar hash e controlar sessão. | DataEncryptionService e serviços relacionados. |
| exception / validation | Padronizar falhas e validar dados. | GlobalExceptionHandler e CpfValidator. |

## Conceitos de orientação a objetos

O encapsulamento aparece nos atributos privados dos modelos e nos métodos de acesso gerados com Lombok. O Service verifica as condições antes de salvar o estado. DTOs limitam os dados transferidos: AdministrationResponseDTO entrega identificação e e-mail, sem o hash da senha.

A herança é utilizada em Registrated, classe abstrata com @MappedSuperclass. Company, Store, Employee, EmployeeRole, SubscriptionPlan e UserAccount herdam situação e datas de controle. Address e Administration têm mapeamentos próprios. Registrated não cria tabela separada.

As interfaces DAO abstraem a persistência ao estender JpaRepository, cuja implementação é fornecida pelo Spring. CpfValidator implementa ConstraintValidator e o interceptor implementa HandlerInterceptor. Esses contratos permitem integrar comportamentos próprios aos pontos de extensão dos frameworks.

A composição e a injeção por construtor permitem que EmployeeService utilize DAOs, DataEncryptionService e DataAnonymizationService sem concentrar todas as tarefas na entidade. A implementação da cifra fica centralizada no serviço responsável.

## Entidades e relacionamentos

O domínio contém Company, Store, Address, Employee, EmployeeRole, UserAccount, SubscriptionPlan e Administration. Company referencia SubscriptionPlan; Store referencia Company e Address; Employee referencia Store e EmployeeRole; UserAccount referencia Employee.

Figura 1 – Relações mapeadas no domínio administrativo.

SubscriptionPlan 1 ← N Company 1 ← N Store 1 ← N Employee. Address 1 ← 1 Store; EmployeeRole 1 ← N Employee; Employee 1 ← 1 UserAccount. Administration possui credenciais próprias.

## Criptografia AES e hash BCrypt

DataEncryptionService usa AES/GCM/NoPadding com chave de 256 bits, IV aleatório de 12 bytes e tag de autenticação de 128 bits. O construtor decodifica a chave Base64 de mottainai.crypto.key, alimentada por MOTTAINAI_AES_KEY, e rejeita tamanho diferente de 32 bytes.

encrypt converte o texto em UTF-8, gera o IV e retorna Base64 de IV seguido do conteúdo cifrado e da tag. decrypt separa essas partes e verifica a autenticação antes de retornar o texto. Entradas nulas retornam null; falhas de formato, chave ou integridade geram exceção técnica.

EmployeeService cifra CPF, nome, e-mail e telefone. CompanyService e StoreService cifram contatos; UserAccountService cifra o e-mail. Administration mantém o e-mail utilizado na consulta exata de login. O projeto não cifra indistintamente todos os campos das tabelas.

Figura 2 – Fluxo da proteção no cadastro de funcionário.

EmployeeRequestDTO → EmployeeService → DataEncryptionService.encrypt → Employee → EmployeeDao.save → PostgreSQL. Na consulta, o Service usa decrypt para preencher o DTO de resposta.

PasswordHashService utiliza BCryptPasswordEncoder para gerar o hash e comparar a senha com matches. É utilizado por Administration e UserAccount. A senha não é descriptografada. AdministrationService limita a senha a 72 bytes em UTF-8.

### Exclusão lógica e histórico

Os Services de empresa, loja, funcionário, cargo, conta e plano utilizam exclusão lógica e regras próprias de substituição de dados. Active=false representa situação inativa; DeletedAt identifica exclusão. Os estados não são equivalentes. Address também recebe tratamento de exclusão e anonimização.

Na exclusão de Employee, CPF, e-mail e telefone são substituídos por marcadores ANON_; a data de contratação é substituída e a conta vinculada é desativada e anonimizada. O nome permanece cifrado para identificação histórica. O processo não remove todos os dados pessoais. Loja e cargo passam a registros técnicos que preservam os vínculos obrigatórios.

Administration possui exclusão física específica. O Service recebe o ID do solicitante obtido da sessão e bloqueia a exclusão da própria conta. Excluir outro administrador é permitido. A sessão da conta removida é invalidada no próximo acesso protegido.

### Autenticação e integração com JSP

AuthenticationController recebe POST /login. AdministrationAuthService consulta AdministrationCredentialsDao e valida a senha com BCrypt. Após sucesso, invalida a sessão anterior e cria outra com administrationId e administrationEmail. O logout invalida a sessão.

AdministrationSessionInterceptor verifica os atributos e a existência do administrador. Sem acesso válido, redireciona para /login. As páginas estão em src/main/webapp/WEB-INF/jsp/ e os estáticos em src/main/resources/static/assets/.

ViewFilterUtils filtra dados preparados para as telas. ViewPaginationUtils limita a tabela a seis registros, ordena os itens quando recebe o identificador e ajusta páginas fora do limite. Essa paginação recorta uma lista em memória, sem representar paginação SQL dos CRUDs.

### Implementação entregue

A implementação entregue contém as camadas administrativas, páginas JSP, persistência JPA, consulta JDBC de autenticação, validações, proteção dos dados e testes automatizados. As classes e os registros de execução estão identificados nas seções de requisitos, testes e evidências.

## 4. Requisitos e entregáveis

| ID | Requisito / entregável | Local no repositório | Status |
|---|---|---|---|
| R1 | CRUD de oito recursos. | controller/ e service/ | Implementado |
| R2 | JPA e login JDBC. | dao/ e security/ | Implementado |
| R3 | DTOs, validações e exceções. | dto/, validation/ e exception/ | Implementado |
| R4 | JSP integrado ao backend. | WEB-INF/jsp/ e ViewControllers | Implementado |
| R5 | AES, BCrypt e sessão. | security/ | Implementado |

| ID | Requisito / entregável | Local no repositório | Status |
|---|---|---|---|
| R6 | Bloqueio de autoexclusão. | AdministrationService | Implementado |

Exemplo de contrato: /api/companies recebe POST para criar e GET para listar. /api/companies/{companyId} recebe GET, PUT e DELETE para consultar, atualizar e excluir. Employee e Store possuem controladores equivalentes. Os formulários JSP são tratados pelos ViewControllers.

# 5. Validação e testes

## Critérios de validação

- Controllers devem validar entradas e encaminhar operações aos Services.
- Vínculos obrigatórios e proibições de edição e exclusão devem ser respeitados.
- AES deve recuperar dados e rejeitar conteúdo alterado; hashes não devem sair nos DTOs.
- Login deve criar sessão e autoexclusão deve ser bloqueada no backend.
- As tabelas das telas devem respeitar seis registros por página.

## Casos de teste

| ID | Cenário | Resultado esperado | Evidência de referência |
|---|---|---|---|
| T1 | Cifra e alteração do conteúdo. | Recuperar original e rejeitar adulteração. | Cenário em DomainRegressionTest. |
| T2 | Login administrativo. | Validar BCrypt sem retornar hash. | Cenário em AdministrationAuthServiceTest. |
| T3 | Excluir a própria conta. | Bloquear parâmetros forjados. | Cenário em AdministrationDeletionTest. |
| T4 | Sessão de conta removida. | Revogar no próximo acesso. | Cenário em AdministrationDeletionTest. |
| T5 | Paginação e busca. | Filtrar e limitar a lista. | CrudPaginationTest e CrudSearchReviewTest. |
| T6 | Fluxo com banco real. | Verificar persistência e rollback. | Condicionado ao ambiente. |

## Registros de execução dos testes

O relatório build/manual/test-summary.txt contém 55 testes encontrados, 54 executados e aprovados, 1 ignorado e nenhuma falha. O arquivo build/documentation-final-tests.log registra uma execução separada com 28 testes aprovados. Esses números descrevem execuções distintas, sem total acumulado. Os códigos dos testes estão em src/test/java/com/mottainai/crudinicial/.

O escopo do resumo local inclui testes de regras e requisições com recursos simulados. O fluxo FinalFrontendLiveIntegrationTest depende de MOTTAINAI_LIVE_TESTS=true e de PostgreSQL configurado; sua execução não está comprovada nesse resumo. CrudInicialApplicationTests depende do banco e não integra a seleção de scripts/LocalTestRunner.java.

## 6. Configuração e execução

| Passo | Ação |
|---|---|
| 1 | Instalar JDK 21 e abrir o projeto Gradle no IntelliJ IDEA ou no terminal. |
| 2 | Preparar PostgreSQL com tabelas compatíveis com os modelos e registros técnicos de anonimização. Hibernate usa ddl-auto=validate. |
| 3 | Definir variáveis do banco e a chave AES no ambiente. O perfil dev já está indicado em application.properties. |
| 4 | Executar .\gradlew.bat bootRun no Windows. Para testes, executar .\gradlew.bat test com o ambiente configurado. |
| 5 | Abrir http://localhost:8080/login e autenticar uma conta Administration previamente cadastrada. |

| Variável / configuração | Finalidade | Exemplo seguro |
|---|---|---|
| DB_HOST / DB_PORT | Servidor e porta PostgreSQL. | Definidos no ambiente |
| DB_NAME | Nome do banco. | Definido no ambiente |
| DB_USER / DB_PASSWORD | Credenciais da conexão. | Definidas no ambiente |
| MOTTAINAI_AES_KEY | Base64 de chave AES de 32 bytes. | Definida no ambiente |
| SPRING_PROFILES_ACTIVE | Seleção do perfil Spring. | dev |
| MOTTAINAI_LIVE_TESTS | Habilita testes com banco real. | true no ambiente de teste |

A conexão utiliza sslmode=require. A configuração valida o schema existente, sem criar ou alterar tabelas. A execução pressupõe tabelas compatíveis com os modelos e registros técnicos de anonimização previamente cadastrados. Os dados cifrados exigem a mesma chave para recuperação. As credenciais do banco e a chave AES são fornecidas pelo ambiente.

## 7. Evidências de entrega

Os caminhos relativos das classes Java abaixo partem de src/main/java/com/mottainai/crudinicial/. Os testes ficam em src/test/java/com/mottainai/crudinicial/ e os relatórios de execução estão em build/.

| Evidência | Link ou local | Status |
|---|---|---|
| Classes e camadas | model/, dto/, dao/, service/ e controller/ | Presentes |
| Segurança e exclusão | security/ e service/EmployeeService.java | Presentes |
| Interface web | src/main/webapp/WEB-INF/jsp/ | Presente |
| Testes e resultado local | src/test/java/ e build/manual/test-summary.txt | Disponíveis |

# 8. Conclusão

O CRUD administrativo aplica orientação a objetos na separação entre modelos, DTOs, Controllers, Services e DAOs. A solução integra páginas JSP à persistência PostgreSQL e centraliza validações, tratamento de exceções e regras de negócio.

A proteção dos dados utiliza AES para campos recuperáveis, BCrypt para senhas e controle de sessão administrativa. As regras de exclusão preservam o histórico conforme cada recurso e bloqueiam a autoexclusão de administradores. Os resultados de testes documentados correspondem às execuções registradas nos relatórios identificados neste documento.

## Histórico de versões

| Versão | Data | Autor | Alterações |
|---|---|---|---|
| 1.0 | 27/10/2026 | Equipe POO/LPR | Documento inicial do CRUD. |
| 1.1 | 29/10/2026 | Equipe POO | Continuação com arquitetura, segurança, JSP e testes. |

## Referência de demanda

Projeto Interdisciplinar – Solicitações por Disciplina, 2026. Itens: CRUD Java, POO, PostgreSQL, interface web, validação e documentação.